

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторные работы по курсу «Информационный поиск»

Студент: В. Д. Маношин
Преподаватель: А. А. Кухтичев
Группа: М8О-410Б
Дата:
Оценка:
Подпись:

Москва, 2025

1 Описание

Целью лабораторных работ по курсу «Информационный поиск» является разработка полноценной поисковой системы для тематического корпуса документов, извлечённых из открытых источников. В рамках проекта реализован полный цикл обработки данных: от сбора и предобработки до построения индексов, выполнения запросов и предоставления пользовательского интерфейса.

Система ориентирована на анализ вопросов и ответов с платформы **Stack Exchange**, в частности — с разделов `ai.stackexchange.com` и `datascience.stackexchange.com`. Эти источники выбраны благодаря их высокому качеству контента, технической направленности и актуальности для специалистов в области информационных технологий и анализа данных.

Этапы реализации

Работа включает следующие ключевые компоненты:

- **Сбор данных:** автоматизированный загрузчик извлекает архивы `.7z` с сайта Archive.org, распаковывает XML-файлы (`Posts.xml`) и выбирает только основные сообщения (вопросы) с достаточным объёмом текста (не менее 100 слов).
- **Хранение корпуса:** каждый документ сохраняется как отдельный текстовый файл в директории `corpus/`, а метаданные (URL, заголовок, хеш содержимого, время загрузки) — в JSON-файле с тем же идентификатором.
- **Токенизация:** текст приводится к нижнему регистру, разбивается на слова, удаляются все неалфавитно-цифровые символы. Токены короче двух символов отбрасываются.
- **Стемминг:** реализован алгоритм, вдохновлённый Porter Stemmer, для нормализации слов перед индексацией. Это позволяет объединять формы одного слова (например, `running` → `run`).
- **Построение индексов:**
 - Обратный индекс (`inverted_index.bin`) — отображает термин в список ID документов, где он встречается.
 - Прямой индекс (`forward_index.bin`) — хранит информацию о каждом документе: ID, заголовок, URL.

Индексы сериализуются в бинарном виде для компактности и быстрого чтения.

- **Булев поиск:** реализована поддержка сложных запросов с операторами `&&` (AND), `||` (OR), `!` (NOT) и скобочной группировкой. Поиск выполняется за миллисекунды благодаря эффективному слиянию отсортированных списков документов.
- **Веб-интерфейс:** с помощью фреймворка Flask реализован веб-сервер, позволяющий вводить запросы через браузер, просматривать результаты и использовать постраничную навигацию.

Ключевые особенности

- Все компоненты написаны на языках **Python** (сбор данных, веб-интерфейс) и **C++** (токенизация, стемминг, индексация, поиск) без использования сторонних библиотек, кроме минимально необходимых (`std::vector`, `std::string`, `py7zr`, `Flask`).
- Система обеспечивает **идемпотентность**: повторная загрузка не дублирует данные, проверяется хеш содержимого.
- Используется **вежливое поведение** при скачивании: задержка между запросами, HEAD-запросы для проверки изменений.
- Поддерживается масштабируемость: система способна обрабатывать десятки тысяч документов с высокой скоростью поиска.

Статистика корпуса

В результате работы сформирован корпус из **30 000 документов** (по 15 000 на источник), средний размер документа — около **500–2 000 слов**. Документы прошли полную очистку: удалены HTML-теги, мусорные символы, сохранён только семантически значимый текст.

2 Исходный код

Реализация состоит из двух частей: **сбор данных и веб-интерфейс (Python)** и **обработка/поиск (C++)**. Все компоненты разработаны с акцентом на производительность, простоту и минимальные зависимости. Система не использует STL за пределами `std::string` и `std::vector`, а также сторонние поисковые библиотеки.

Сбор данных (`crawler.py`)

Модуль загружает архивы `.7z` с сайта Archive.org, содержащие дампы сообществ Stack Exchange:

- `ai.stackexchange.com`
- `datascience.stackexchange.com`

После скачивания архив распаковывается с помощью библиотеки `py7zr`, извлекается файл `Posts.xml`. Затем парсер `xml.etree.ElementTree` проходит по всем записям типа `PostTypeId="1"` (вопросы), извлекая заголовок и тело сообщения.

HTML-разметка очищается с помощью регулярных выражений:

Текст объединяется: `title + "\n\n" + body`, и проверяется на минимальную длину — не менее 100 слов. Каждый документ сохраняется в директорию `corpus/` как `<source>_<id>.txt`, а метаданные — в JSON-файл с тем же именем, включая:

- URL статьи;
- Хеш содержимого (MD5);
- Время извлечения;
- Количество слов.

Для обеспечения идемпотентности повторная загрузка не перезаписывает существующие файлы. Между запросами соблюдается задержка 1 секунда.

Токенизатор (`tokenizer.cpp`)

Работает в два этапа:

1. Чтение всего файла в память через `std::ifstream` в двоичном режиме.

2. Посимвольная обработка:

- Латинские буквы приводятся к нижнему регистру;
- Допустимыми символами считаются только [a-z0-9];
- Прочие символы являются разделителями.

Накопленное слово добавляется в список, если его длина 2. Результат — вектор нормализованных терминов, готовых к стеммингу и индексации.

Стеммер (PorterStemmer.h)

Реализован упрощённый алгоритм Портера на C++. Основные шаги:

- Удаление суффиксов: `s`, `ss`, `ies`, `ed`, `ing`;
- Правила замены: `ational` → `ate`, `tional` → `tion`, `ization` → `ize`;
- Удаление окончаний: `al`, `ance`, `ence`, `er`, `ic`;
- Коррекция: `e` удаляется при определённых условиях;
- Удвоение согласных: например, `ll` после `e` может быть сокращено до `l`.

Функция `stem()` принимает строку, приводит её к нижнему регистру и последовательно применяет правила. Алгоритм работает за $O(k)$ для слова длины k .

Построение индексов (index.cpp)

Обрабатываются все файлы в директории `corpus/`. Для каждого документа:

- Извлекаются токены;
- Каждый токен стеммируется;
- Формируется обратный индекс: `term` → `[doc_id]`.

Также строится прямой индекс: `doc_id` → `title`, `url`, где URL формируется автоматически на основе имени файла.

Индексы сериализуются в бинарные файлы:

- `inverted_index.bin`: сначала количество терминов, затем для каждого:
 - Длина строки (4 байта);

- Само слово;
- Количество документов (4 байта);
- Список ID документов (по 4 байта каждый).
- `forward_index.bin`: аналогично — число документов, затем для каждого:
 - `doc_id`, длина и текст заголовка, длина и URL.

Перед сохранением обратный индекс сортируется по алфавиту. Это позволяет использовать бинарный поиск при выполнении запросов.

Булев поиск (`search.cpp`)

Поддерживается полный синтаксис:

- `car engine` → `car && engine`
- `car || truck`
- `!bike`
- `(car || truck) && engine`

Архитектура:

1. Загрузка индексов из бинарных файлов;
2. Лексический анализ: входной запрос разбивается на токены (слова, `,`, `||`, `!`, скобки);
3. Синтаксический анализ методом рекурсивного спуска:
 - `eval_expr()` — обрабатывает OR;
 - `eval_term()` — AND;
 - `eval_factor()` — NOT и скобки.
4. Операции над списками:
 - AND: слияние двух отсортированных списков за $O(|A| + |B|)$;
 - OR: объединение с удалением дубликатов;
 - NOT: дополнение до полного множества документов.

Поиск выполняется за доли миллисекунды даже на корпусе из 30 000 документов.

Веб-интерфейс (`web.py`)

Для удобства используется веб-интерфейс на Flask. Он:

- Отображает форму ввода запроса;
- При поиске запускает внешнюю программу `./searcher "запрос"` через `subprocess.run()`;
- Парсит вывод: каждая строка вида `title | url` преобразуется в ссылку;
- Поддерживает постраничную навигацию (по 50 результатов на страницу).

Интерфейс работает на `http://localhost:5000`, имеет чистый, отзывчивый дизайн и полностью автономен.

Производительность:

- Время ответа: $\sim 1\text{--}5$ мс;
- Поддержка сложных логических выражений;
- Масштабируемость: легко адаптируется под новые источники.

3 Выводы

Выполнив лабораторные работы по курсу «Информационный поиск», я получил полное практическое понимание архитектуры поисковой системы — от сбора данных до пользовательского интерфейса. Реализация всех компонентов «с нуля» позволила глубоко разобраться в деталях, которые скрыты при использовании готовых решений, таких как Elasticsearch или Whoosh. Во-первых, я освоил современные подходы к сбору данных: вместо медленного обхода веб-страниц я научился работать с готовыми дампами с Archive.org, эффективно распаковывая архивы .7z и извлекая структурированные данные из XML. Это показало, что для многих задач не нужно писать сложного краулера — достаточно использовать открытые источники с качественным контентом, такими как Stack Exchange. Во-вторых, реализация токенизатора и стеммера на C++ без использования STL (кроме `std::string` и `std::vector`) позволила оценить важность контроля над производительностью и памятью. Я лучше понял, как устроены базовые операции: чтение файла в память, нормализация текста, разбиение на слова. Стеммер, вдохновлённый алгоритмом Портера, продемонстрировал, насколько даже простые правила могут эффективно нормализовать лексику и повысить точность поиска. В-третьих, построение бинарных индексов (`inverted_index.bin`, `forward_index.bin`) показало, насколько важна компактность и скорость доступа. Хранение данных в сериализованном виде позволяет быстро загружать индексы при старте программы и минимизирует использование памяти. Сортировка терминов по алфавиту открывает возможность для будущего улучшения — например, бинарного поиска или использования trie. Наконец, реализация булева поиска с поддержкой `&&`, `||`, `!` и скобок показала силу алгоритмов слияния отсортированных списков. Даже на корпусе из 30 000 документов время ответа составляет доли миллисекунды. Интеграция с веб-интерфейсом через Flask превратила консольную утилиту в удобный сервис, доступный из любого браузера, с постраничной навигацией и чистым дизайном. Полученный опыт имеет прямое применение в реальных проектах: от корпоративного поиска до анализа технической документации и создания ассистентов. Теперь я не просто использую поисковые технологии — я понимаю, как они устроены внутри, и могу адаптировать их под конкретные задачи.

Список литературы

- [1] Маннинг, К. Д., Рагхаван, П., Шютце, Х. *Введение в информационный поиск* — М.: Издательский дом «Вильямс», 2011. — 528 с. ISBN 978-5-8459-1623-4 (рус.). Перевод с английского под ред. доктора физ.-мат. наук Д. А. Ключиной.
- [2] Porter, M. F. *An algorithm for suffix stripping* // Program. — 1980. — Vol. 14, No. 3. — P. 130–137.
- [3] Zipf, G. K. *Human Behavior and the Principle of Least Effort* — Cambridge, MA: Addison-Wesley, 1949.
- [4] Wikipedia contributors. *Zipf's law* // Wikipedia, The Free Encyclopedia. — URL: https://en.wikipedia.org/wiki/Zipf%27s_law (дата обращения: 30.12.2025).
- [5] ГОСТ Р 7.0.5–2008. *Библиографическая ссылка. Общие требования и правила составления*. — Введ. 2008–07–01. — М.: Стандартинформ, 2008. — 33 с. URL: <http://www.ifap.ru/library/gost/7052008.pdf> (дата обращения: 30.12.2025).